TECHNICAL GAP ANALYSIS · BUILD PLAN · FEASIBILITY

Drivly

vs the Client Requirement

Target product: an Automated Car-Sharing Platform — Free-Floating & Station-Based

One-line finding

The client asked for an **operator-owned, automated, keyless IoT car-sharing platform**; what was built (Drivly) is a **peer-to-peer rental marketplace**. They share a large supporting base, but the keyless / IoT / geofencing core that defines the client product is the real remaining work — and roughly 90–95% of it can be built without hardware.

57	~40–45%	90–95%	$7.5k
atomic requirements (50 Must · 7 Should)	complete toward the target product	buildable & demoable without hardware	fixed dev price 12–16 week build

Status legend

DONE	PARTIAL	MISSING	REPURPOSE
built & usable	exists, needs rework	not built	wrong model, re-aim

Effort legend

S	M	L	XL
~1–3 dev-days	~4–8 dev-days	~2–3 dev-weeks	~4+ dev-weeks

Prepared for: Anouar (client)

Source: rent-a-car-system.docx (SRS) · June 26, 2026

Contents

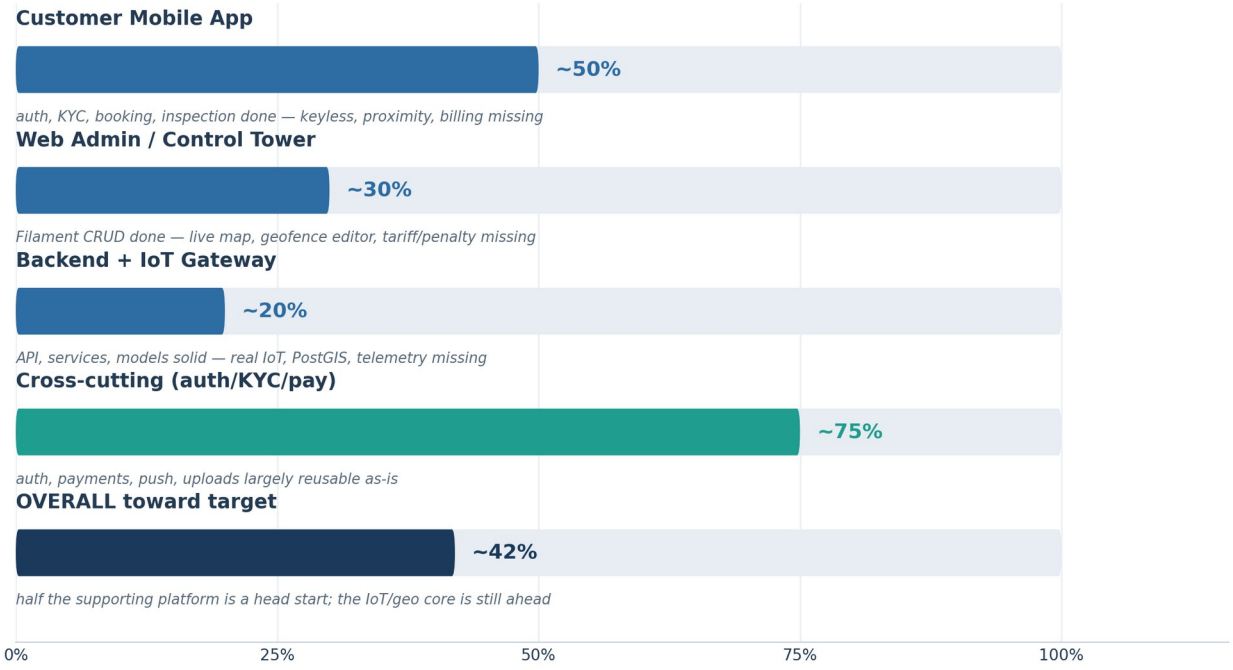
1. Executive Summary

We compared the **client requirement** for an Automated Car-Sharing Platform (operator-owned fleet, fully contactless, IoT keyless cars, GPS geofencing) against **Drivly**, the application built so far. The single most important finding is a **business-model mismatch**: Drivly is a peer-to-peer rental marketplace (Turo / Getaround style), while the client asked for a single operator that owns the whole fleet and runs it with zero on-site staff via IoT-enabled keyless cars.

This is not a missing-features problem; it is a different product aimed at a different owner. The good news: the **supporting half** of the platform — login/KYC, payments, the booking wizard, the map and discovery screens, trip start/end, photo inspections, disputes, and the admin panel — is genuinely built and reusable. So this is a strong head start, not a restart. Roughly 90–95% of the target can be built and demoed now without any hardware, using a "swap-in" gateway adapter (a software car simulator today; the real vendor plugged in later).

Where we stand — progress toward the target product

These bars show how far the existing code carries us *toward the new operator-owned target* — not how "finished" Drivly is. Drivly is a working app; it is simply aimed at a different model.



Each pillar scored against the target. The on-device "GPS control center" is fully simulated — there is no real IoT, geofencing, or proximity gating behind it today.

Bottom line

Question	Honest answer
Is it the same product?	No — Drivly is a peer-to-peer marketplace; the target is an operator-owned, zero-human, IoT keyless fleet.
How much is reusable?	Roughly half — auth/KYC, payments/wallet, discovery + map, booking wizard, inspections, disputes, admin, and push all carry over.

Question	Honest answer
Biggest risk	The vendor GPS/IoT API and a test device — until they arrive, the keyless/telemetry path cannot be validated against real hardware.
Can it be built without hardware?	Yes — ~90–95% via a VehicleGateway adapter with a software simulator (the existing on-device GPS prototype is an early version of this idea).
Realistic timeline	The proposal's 11–12 weeks is optimistic for the full target; expect a strong simulator-backed build, with real-hardware integration likely pushing toward 14–16 weeks.
Realistic cost framing	The $7,500 / 5-milestone structure is a reasonable baseline, but the model change and IoT/geofencing scope warrant revisiting the M4 effort honestly rather than treating it as "wiring."
Hard dependency	Vendor name + full API docs + test-device login by ~end of Week 1, or milestone M4 (IoT/payments/pricing) slips by the same delay.

Top 5 things to build next

1. **IoT VehicleGateway** **[MISSING]** — one interface (unlock / lock / immobilize / status+telemetry) with a software simulator now and the real vendor adapter later.
2. **Geofencing engine (PostGIS)** **[MISSING]** — operational zones + drop-off zones, out-of-bounds detection, and the ≤ 15 m proximity gate for unlock.
3. **Keyless flow rework** **[PARTIAL]** — proximity-gated "Unlock & Start," strict 4-photo walkaround enforcement, and pre-lock automated checks (in-zone, ignition off, doors shut).
4. **Live Control Tower** **[MISSING]** — admin fleet map with color-coded statuses (Green / Blue / Red / Orange) fed by 60 s telemetry.
5. **Deposit holds + dynamic pricing/penalty matrix** **[MISSING]** — Stripe pre-auth holds and forfeiture, plus hourly/daily/seasonal tariffs with grace-period late-return auto-billing.

2. Two Different Products: Asked vs Built

The most important thing to understand before any feature list: the document you commissioned and the app that was built describe **two different businesses that happen to share the same screens.** Both are “car rental apps,” but they sit at opposite ends of the rental world.

- **What you asked for:** an operator-owned, fully automated, keyless car-sharing fleet — one company owns every car, and a customer can walk up, unlock with their phone, drive, and drop off with zero human involvement.
- **What was built (Drivly):** a peer-to-peer (P2P) marketplace — many private owners (“hosts”) list their own cars and renters book them, much like Turo.

2.1 Side-by-side: the two business models

Dimension	Client requirement — Automated free-floating	What was built — Drivly P2P marketplace
Who owns the cars	One operator owns and manages the entire fleet.	Many independent private owners (“hosts”), each owning their own car.
Who lists them	Nobody self-lists — the operator (admin) adds vehicles in the back-office, including each car’s IoT device IMEI.	Hosts self-list cars (make/model/price/photos), then admin approves.
Human at handover?	**None. 100% contactless, zero on-site staff — the defining promise.**	Manual / owner-mediated handover is the implicit norm; no keyless layer exists.
How you get the key	Phone unlocks the car remotely when within ≤ 15 m; physical key waits in the glovebox; engine started by the customer.	Out of scope as built — key exchange happens off-app between host and renter.
Billing model	On-demand and fine-grained: counter starts at unlock and stops at lock (per-minute / hour / day), plus grace periods and late penalties.	Daily-rental oriented: a booking spans whole days; extension is measured in days, not minutes.
Core tech moat	Keyless IoT (unlock / lock / immobilize + 60 s telemetry) and geofencing (PostGIS zones, ≤ 15 m gate, out-of-bounds detection).	Listing/marketplace mechanics: search, reviews, host earnings, host↔renter chat. No IoT; no geofencing.
Real-world analogue	Zipcar / Share Now / Free2Move / Getaround-Connect.	Turo / classic Getaround.
Trust & safety	Remote immobilize for theft, geofence/border alerts + SMS, deposit hold with forfeiture, cross-user damage comparison.	Host verification, KYC review, disputes, reviews — reputation-based, not device-and-geofence based.
Revenue flow	All revenue flows to the single operator; no third-party payouts.	Renter pays → platform takes a cut → host receives a payout (earnings, wallet, withdraw).

2.2 Why this matters for the code, not just the pitch

The model mismatch determines which existing code is an asset, which must be re-aimed, and which must be deleted. These P2P concepts exist only because Drivly is a marketplace:

Built concept	Why it exists in Drivly (P2P)	Fate under the Automated Platform
Hosts, host roles, host app screens	A marketplace needs sellers	**REPURPOSE** — becomes the single operator's fleet-management role
Host payouts / earnings / wallet withdraw	Owners must be paid	**REPURPOSE** — operator owns all revenue; payout logic removed
Host verification	Vet private owners	**REPURPOSE** — folded into operator onboarding, not customer-facing
Host↔renter chat	Two parties must coordinate	**REPURPOSE** — at most an optional support channel
Instant / request booking	Host accepts/declines requests	**REPURPOSE** — "take the car now" or a simple time-slot reservation
Keyless IoT, geofencing, Control Tower map	Not part of a P2P marketplace	**MISSING** — these define the client product and were never built

> **The prototype is a convincing demo, not a working gateway**
>
> The on-device GPS control screen is the one place the client's core idea appears — and its own header comment admits the device state is "simulated on-device so the whole flow is demoable before the third-party GPS API is wired up." There is no backend gateway, no real IoT, and no real geofencing or proximity gating behind it.

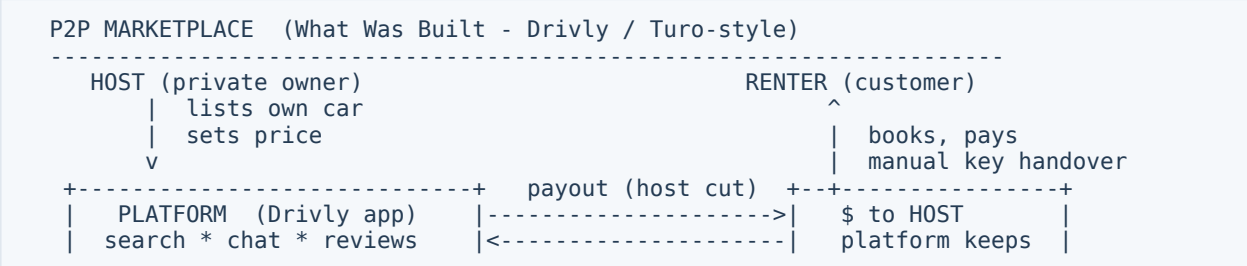
2.3 But the overlap is large — a pivot, not a restart

Both products are car-rental apps, and roughly half of the supporting platform is genuinely reusable today: account creation (email / OTP / social), KYC upload + admin review, Stripe payments and wallet, car discovery with map and filters, the booking wizard (dates → pricing → pay), pre/post-trip inspection with photo upload (which maps almost exactly onto the required 4-photo Digital Walkaround), the trip lifecycle, disputes, reviews, settings, push notifications, and the Filament admin.

A fair estimate: the shared foundation already covers on the order of **45–55%** of the supporting (non-differentiating) work. The remaining effort concentrates in the differentiating layer the requirement is actually about. **Caveat:** Stripe, Firebase, Apple/Google sign-in, and Maps currently run in a credential-free demo mode, and the map screen renders a styled placeholder rather than live Google Maps tiles until a real key is supplied — so "works" means the flows are built, not that they are wired to live services.

2.4 The two value chains at a glance

Follow the car and the money through each model. The left chain has three parties and a commission split; the right has one owner, a device on every car, and software where the human used to be.



```
 |  host verify * disputes     |    renter payment    |   a commission    |
 +-----------------------------+                      +-------------------+
 No IoT. No geofencing. Key changes hands between two people.


AUTOMATED FREE-FLOATING FLEET  (What Was Asked - Zipcar / Share Now-style)
-------------------------------------------------------------------------
   OPERATOR / FLEET (single owner, owns every car + IoT box)
       |  adds cars + IMEI in Control Tower
       v
 +-----------------------------+   unlock/lock/immobilize   +----------------+
 |  PLATFORM + IoT GATEWAY     |--------------------------->|  CAR Smart Box |
 |  geofencing (PostGIS)       |<---------------------------|  GPS/fuel/odo  |
 |  live map * deposit hold    |   60s telemetry            +----------------+
 +--------------+--------------+
                |  proximity-gated unlock (<=15 m), per-min/hour billing
                v
           DRIVER (customer)  -- walks up, taps Unlock, drives, taps Lock
 All revenue -> the OPERATOR. Zero human at handover.
```

3. Full Requirements Inventory

This breaks the client's brief into a numbered, atomic list of requirements — the shared spine of the build plan. Each ID prefix says which part of the system owns it: **MOB-** (mobile app), **ADM-** (web admin / Control Tower), **BE-** (backend, IoT, geofencing), and **NFR-** (non-functional).

> **How to read the "Hardware-dependent?" column**
>
> **No** = build and fully test today against a software simulator. **Partial** = most of it builds now; final validation needs the vendor device. **Yes** = cannot be confirmed until real hardware + the vendor API are in hand. Priority uses MoSCoW (Must / Should).

3.1 Customer Mobile App (MOB-)

The app the renter holds: sign up and prove identity, find and pay for a nearby car, walk to it, unlock it with no staff present, drive, then return and lock it to stop the meter.

ID	Requirement	Source	Priority	Hardware?
MOB-01	Account creation via email, phone, or social login (Apple / Google)	SRS 3.1	**Must**	**No**
MOB-02	Phone number verified by OTP (one-time code)	SRS 3.1	**Must**	**No**
MOB-03	Mandatory KYC: upload Government ID / Passport and Driver License (two documents)	SRS 3.1	**Must**	**No**
MOB-04	Payment setup: secure tokenization of credit / debit cards	SRS 3.1	**Must**	**No**
MOB-05	Digital wallet support (Apple Pay / Google Pay)	SRS 3.1	Should	**No**
MOB-06	Hard block: car cannot be booked without a verified payment method on file	SRS 3.1	**Must**	**No**
MOB-07	Proximity map of available cars within walking distance	SRS 3.2	**Must**	**No**
MOB-08	Each map car shows real-time fuel level, vehicle model, and distance to user	SRS 3.2	**Must**	**Partial**
MOB-09	Booking Step 1: select vehicle + rental duration (start / end date & time)	SRS 3.2	**Must**	**No**
MOB-10	Booking Step 2: add-ons — Premium Insurance, Child Seat, Extra Mileage Package	SRS 3.2	**Must**	**No**
MOB-11	Booking Step 3: authorize security deposit (held funds) + process upfront payment	SRS 3.2	**Must**	**No**
MOB-12	Pedestrian / walking navigation to the precise car GPS location	SRS 3.3	**Must**	**No**
MOB-13	Proximity-gated unlock: button disabled until phone GPS is ≤ 15 m from car's live GPS	SRS 3.3	**Must**	**Partial**
MOB-	Digital Walkaround: 4 photos (front, back, left, right) before	SRS 3.3	**Must**	**No**

ID	Requirement	Source	Priority	Hardware?
14	unlock			
MOB-15	Tap Unlock → trigger sent to IoT gateway → doors unlock	SRS 3.3	**Must**	**Yes**
MOB-16	After unlock, app instructs user to retrieve physical key from glovebox	SRS 3.3	**Must**	**No**
MOB-17	Rental time counter officially begins at the moment of unlock	SRS 3.3	**Must**	**No**
MOB-18	Return: user must park inside a valid geofenced drop-off zone shown on the map	SRS 3.4	**Must**	**Partial**
MOB-19	Drop-off checklist: “keys left in glove compartment” + “belongings collected”	SRS 3.4	**Must**	**No**
MOB-20	“Lock & End Rental” — only after IoT confirms physical lock does the counter stop & invoice generate	SRS 3.4	**Must**	**Yes**

3.2 Web Admin / Control Tower (ADM-)

The operator’s command center in a browser — watch every car live, configure zones and prices, approve users, and handle damage disputes, all without anyone visiting a car.

ID	Requirement	Source	Priority	Hardware?
ADM-01	High-frequency live tracking map of every car in the fleet (the “Control Tower”)	SRS 4.1	**Must**	**Partial**
ADM-02	Color-coded statuses: Available=Green, Rented=Blue, Out-of-Bounds=Red, Maint/Low-Fuel=Orange	SRS 4.1	**Must**	**Partial**
ADM-03	Fleet matrix to add / edit vehicles: brand, model, plate, and IMEI of the IoT device	SRS 4.2	**Must**	**No**
ADM-04	Automated maintenance logs: track mileage via GPS odometer	SRS 4.2	**Must**	**Partial**
ADM-05	Auto-flag vehicles due for oil change, insurance renewal, technical inspection	SRS 4.2	**Must**	**No**
ADM-06	User moderation: review / approve / reject uploaded licenses & KYC	SRS 4.2	**Must**	**No**
ADM-07	“Blacklist” tool to ban fraudulent accounts	SRS 4.2	**Must**	**No**
ADM-08	Tariff manager: dynamic hourly / daily / seasonal pricing per vehicle category	SRS 4.3	**Must**	**No**
ADM-09	Automated grace period (e.g. 15 min) before late penalties apply	SRS 4.3	**Must**	**No**
ADM-10	Late-return auto-billing: auto-bill next full day + admin penalty, fully customizable	SRS 4.3	**Must**	**No**
ADM-11	Cross-user damage comparison: a dent reported on pickup instantly flags previous driver	SRS 4.4	Should	**No**

ID	Requirement	Source	Priority	Hardware?
ADM-12	Deposit forfeiture: review pre/post photos and deduct damage fees from held deposit	SRS 4.4	**Must**	**No**

3.3 Backend, IoT Gateway & Geofencing (BE-)

The invisible engine: speaks to cars through the third-party GPS/IoT vendor, holds the geofence zones, ingests live telemetry every minute, runs safety checks, and triggers theft alarms.

ID	Requirement	Source	Priority	Hardware?
BE-01	Central backend brokering Customer App, Web Admin, and the third-party GPS/IoT API	SRS 2	**Must**	**No**
BE-02	Ingest inbound telemetry: GPS, ignition, central-locking status, fuel %/battery %, odometer	SRS 2	**Must**	**Partial**
BE-03	POST /vehicle/unlock — opens central locking, starts rental counter, app to trip mode	SRS 5	**Must**	**Yes**
BE-04	POST /vehicle/lock — closes locking, stops counter, generates invoice, returns car to map	SRS 5	**Must**	**Yes**
BE-05	GET /vehicle/status polled every 60 s — lat/long, fuel %, odometer; updates live map	SRS 5	**Must**	**Partial**
BE-06	POST /vehicle/immobilize — cuts starter motor relay, vehicle cannot restart, alerts police	SRS 5	**Must**	**Yes**
BE-07	Geofencing engine using PostGIS: operational urban zones + drop-off / station zones	SRS 6 / 3.4	**Must**	**No**
BE-08	Proximity calculation: real distance between phone GPS and car's live GPS for the ≤ 15 m gate	SRS 3.3	**Must**	**Partial**
BE-09	Out-of-bounds detection: is a car inside its allowed zone?	SRS 4.1 / 3.4	**Must**	**Partial**
BE-10	Pre-lock checks before end-rental: in perimeter? ignition off? all doors & windows shut?	SRS 3.4	**Must**	**Yes**
BE-11	Free-floating billing engine: per-minute / hour / day metering tied to unlock/lock events	SRS 3.3 / 3.4	**Must**	**No**
BE-12	Security deposit HOLD / pre-authorization (held, not captured) + later capture or release	SRS 3.2 / 4.4	**Must**	**No**
BE-13	Dead-zone fail-safe: BLE fallback commands directly to the smart box (if hardware supports)	SRS 7	Should	**Yes**
BE-14	Dead-zone fail-safe: emergency offline toll-free verification code	SRS 7	Should	**Partial**
BE-15	Theft mode: car leaves zone / crosses border without active rental → high-priority alert	SRS 7	**Must**	**Partial**
BE-16	Theft mode: automated SMS to operations team	SRS 7	Should	**No**
BE-17	Theft mode: prepare a safe remote engine-kill command for admin confirmation	SRS 7	Should	**Yes**

ID	Requirement	Source	Priority	Hardware?
BE-18	Vendor-agnostic gateway adapter: same backend works against a simulator now, vendor later	SRS 2	**Must**	**No**

3.4 Non-Functional Requirements (NFR-)

Qualities the system must have regardless of any single feature — the database, performance and security, the platforms it ships to, and the languages it speaks.

ID	Requirement	Source	Priority	Hardware?
NFR-01	Database = PostgreSQL with the PostGIS extension for precise geofence boundary lookups	SRS 6	**Must**	**No**
NFR-02	Backend = Laravel, chosen for fast async webhook handling from GPS gateways	SRS 6	**Must**	**No**
NFR-03	Mobile app delivered on both iOS and Android (Flutter or React Native)	SRS 3 / 6	**Must**	**No**
NFR-04	Security: tokenized payment credentials, secure KYC document storage, authenticated APIs	SRS 3.1	**Must**	**No**
NFR-05	Scalability: reliably ingest 60 s telemetry across the whole fleet without backlog	SRS 5	**Must**	**Partial**
NFR-06	Multi-language UI: Arabic, French, English with RTL support (Morocco market)	Proposal	Should	**No**
NFR-07	Bi-directional, real-time data flow between cars, backend, app, and admin map	SRS 2	**Must**	**Partial**

3.5 Inventory count summary

By priority (MoSCoW)

Must (50) Should (7)

By hardware dependency

No — build now (35) Partial — validate later (14) Yes — needs device (8)

```
Total requirements: 57   (MOB 20 | ADM 12 | BE 18 | NFR 7)

By priority:
  Must   = 50   (MOB 19 | ADM 11 | BE 14 | NFR 6)
  Should =  7   (MOB-05 | ADM-11 | BE-13, BE-14, BE-16, BE-17 | NFR-06)
  Could  =  0   (every item the client wrote is at least a Should)
```

```
By hardware dependency:
  No      (build & test fully today)       = 35
  Partial (build now, validate later)      = 14
  Yes     (gated on vendor + device)       =  8   (MOB-15, MOB-20, BE-03, BE-04,
                                                    BE-06, BE-10, BE-13, BE-17)
```

> **One-line count:** 57 atomic requirements — 50 Must / 7 Should / 0 Could. Only 8 are hard hardware-dependent (Yes) and 14 are Partial, meaning **49 of 57 (~86%)** can be built and demonstrated now against a simulator before the GPS/IoT hardware arrives.

4. Gap Analysis — Where We Stand

This maps every requirement onto what Drivly actually contains today. The headline: Drivly is a well-built P2P marketplace; the target is an operator-owned, automated, keyless fleet. The biggest gap is a **business-model mismatch, not a feature count.** Roughly half of the supporting platform is reusable; the half that defines the product — real IoT keyless access, geofencing, live fleet control, deposit holds, on-demand billing — is not built.

4.1 Mobile App (Customer)

ID	Requirement	Status	In Drivly today	Still to build	Eff
MOB-1	Account: email / phone-OTP / social	DONE	Full auth incl. Google, Apple, OTP, reset (demo mode)	Wire live keys; enforce phone-OTP at signup	S
MOB-2	KYC: ID/Passport AND Driver License	PARTIAL	KYC upload + admin review built; photo upload via Spatie	Enforce two mandatory distinct doc types	S
MOB-3	Tokenize card; no booking without method	PARTIAL	Stripe + Wallet; charge/confirm; payment screen (demo)	Save-on-file; enforce method gate; Apple/Google Pay	M
MOB-4	Discovery map: nearby cars, live fuel	PARTIAL	Discovery, maps lib + geolocator present	Styled placeholder w/o key; no live telemetry markers	M HW
MOB-5	Booking Step 1: vehicle + duration	DONE	Booking flow + pricing endpoint (daily dates)	Extend duration model to on-demand / hourly	S
MOB-6	Booking Step 2: add-ons	MISSING	No add-on concept in booking	Add-on catalog + line items + price impact	M
MOB-7	Booking Step 3: deposit HOLD + payment	PARTIAL	Upfront Stripe payment exists	Deposit hold / pre-auth (manual capture) lifecycle	M
MOB-8	Walking navigation to car	MISSING	Map screen exists; no walking mode	Pedestrian routing to precise car GPS	S
MOB-9	Proximity gate (≤ 15 m)	MISSING	Unlock button exists but distance simulated	Real phone-vs-live-car GPS ≤ 15 m enforcement	M HW
MOB-10	Walkaround: 4 photos	PARTIAL	Inspection model + pre/post photo upload	Enforce exactly 4 labeled shots as hard gate	S
MOB-11	Unlock → IoT → key → counter starts	MISSING	Simulated lock/unlock toggle only	Real unlock command; key UX; counter on confirm	M HW
MOB-12	Return inside geofenced drop-off zone	MISSING	No geofence concept anywhere	Drop-off zones + in-zone map validation	M
MOB-13	Pre-lock checks (zone/ignition/doors)	MISSING	None	Check sequence reading live telemetry	M HW
MOB-14	Drop-off checklist	MISSING	None	Simple checklist gate before Lock & End	S
MOB-15	Lock & End → IoT confirm → invoice	PARTIAL	Trip end exists but manual, daily invoice	Lock confirm gate; stop on lock; per-min/hr invoice	M HW

ID	Requirement	Status	In Drivly today	Still to build	Eff
MOB-16	Free-floating per-min / hour billing	PARTIAL	Daily-oriented; extension in whole days	New engine: per-min/hr metering, live cost, take-now	L
MOB-17	Multi-language (AR / FR / EN) + RTL	MISSING	Locale persisted but UI strings English only	i18n extraction + AR/FR + RTL layout	M

4.2 Web Admin — “Control Tower”

ID	Requirement	Status	In Drivly today	Still to build	Eff
ADM-1	Live fleet map, color-coded statuses	MISSING	Filament admin exists; no live map	Tracking map + status colors + telemetry feed	L HW
ADM-2	Fleet matrix w/ brand, model, plate, IMEI	PARTIAL	CarResource CRUD w/ make/model/plate	Add IMEI / IoT device id; fleet vehicle model	M
ADM-3	Automated maintenance logs (odometer)	MISSING	Mileage field; no live odometer or auto-flag	Rules engine on GPS odometer; renewal tracking	M HW
ADM-4	User moderation: approve/reject KYC	DONE	KYC pending/review + Filament; suspend/unsuspend	Confirm review covers ID and license	S
ADM-5	Blacklist tool	PARTIAL	suspend/unsuspend exists	Dedicated hard-ban distinct from suspend	S
ADM-6	Geofencing config (zones)	MISSING	No PostGIS; plain lat/lng columns	PostGIS + polygon zone editor + lookup	L
ADM-7	Dynamic pricing (hourly/daily/seasonal)	MISSING	dynamic_pricing flag only; daily price	Tariff matrix + pricing engine per category	L
ADM-8	Penalty matrix: grace + late auto-billing	MISSING	Trip can be overdue; no auto-billing	Grace period; auto-bill next day + penalty	M
ADM-9	Cross-user damage comparison	MISSING	Disputes exist; no cross-user auto-flag	Compare pre/post photos; flag prior driver	M
ADM-10	Deposit forfeiture from held deposit	MISSING	Dispute resolve exists; no hold to deduct	Deduction workflow tied to MOB-7 hold	M
ADM-11	Admin disputes review	PARTIAL	File + resolve/escalate + Filament resource	Add deposit / photo-comparison tie-ins	S

4.3 Backend, IoT Gateway & Geo

ID	Requirement	Status	In Drivly today	Still to build	Eff
BE-1	Laravel + token auth + admin + queues	DONE	Laravel 12, Sanctum, Filament 3, Horizon, Reverb	Keep as foundation	—
BE-2	PostgreSQL + PostGIS for geofencing	MISSING	Postgres + Redis target; no PostGIS	Enable PostGIS; spatial columns + zone lookups	M
BE-3	IoT gateway: vendor	MISSING	On-device simulation only; no	VehicleGateway + Simulated	L

ID	Requirement	Status	In Drivly today	Still to build	Eff
	adapter		backend gateway	(now) + Vendor (later)	HW
BE-4	POST /vehicle/unlock	MISSING	None on backend	Command endpoint + state transition + counter	M HW
BE-5	POST /vehicle/lock	MISSING	None on backend	Command + lock confirm + invoice + return to map	M HW
BE-6	GET /vehicle/status every 60 s	MISSING	Manual phone POST location; no live feed	60 s telemetry ingest → live store → admin map	L HW
BE-7	POST /vehicle/immobilize	MISSING	Simulated immobilizer toggle only	Real immobilize command + alerting	M HW
BE-8	Live telemetry fields on fleet vehicle	MISSING	Static listing lat/lng; no IMEI/telemetry	Telemetry schema + ingestion + history	M HW
BE-9	Theft mode: alert + SMS + engine-kill prep	MISSING	None	Geofence watcher; alert + SMS; staged kill	M HW
BE-10	Fail-safe: BLE + offline code (dead zones)	MISSING	None	BLE command path + offline verification code	L HW
BE-11	Payments backend: charges + webhook	DONE	Stripe + PaymentService + webhook	Add deposit pre-auth / manual capture	—
BE-12	Push notifications + device tokens	DONE	FCM via Firebase; register/unregister	Reuse for theft / late alerts	—

4.4 Non-Functional / Cross-Cutting

ID	Requirement	Status	Today	To build	Eff
NFR-1	Operator owns the entire fleet (zero-human)	REPURPOSE	P2P: cars host-owned; host roles & verification	Re-aim to operator-owned; remove host ownership	M
NFR-2	Live credentials (Stripe/Firebase/Maps/SMS)	PARTIAL	All run in demo mode without real keys	Wire client-provided live keys	S
NFR-3	Real-time channel for live map / trip	DONE	Reverb WS + Flutter web_socket_channel	Connect to the telemetry stream	S

4.5 The “Connected-Car” screen is a prototype, not working IoT

Brutally clear: none of it talks to a real car

The connected-car control center provides lock/unlock and engine-immobilizer toggles plus a telemetry readout — but every bit of that state is generated on the phone itself: fake telemetry, fake lock state, fake proximity.

Its own comments say it is “the prototype of the client’s core idea … simulated on-device so the flow is fully demoable without the third-party GPS provider’s API.”

There is no backend gateway, no real IoT device, no geofencing, and no real proximity gate behind

> it. It validates the flow and UX (genuinely useful), but it is 0% of the real telematics integration.

```
WHAT THE PROTOTYPE DOES TODAY            WHAT THE TARGET NEEDS
---------------------------              --------------------------------
[Phone UI]                               [Phone UI]
   |  (button tap)                          |  unlock/lock command
   v                                        v
[on-device fake state]  <-- ALL FAKE     [Central Backend / VehicleGateway]
   ^                                        |   (Simulated now / Vendor later)
   |  fake telemetry                        v
(no backend, no car)                     [Third-Party GPS/IoT API] <-> [Car Smart Box]
                                              ^  real GPS/fuel/lock/odometer (60s)
```

4.6 Reuse ledger — every Drivly capability classified

KEEP = use as-is or light tweaks · **REPURPOSE** = re-aim from P2P to operator · **REMOVE** = not needed in the target product.

Drivly capability	Decision	Why	Notes
Email / phone-OTP / social auth	KEEP	Identical need in target	Wire live keys
KYC upload + admin review	KEEP	Target requires ID + license KYC	Enforce both doc types
Stripe payments + webhook	KEEP	Core to target	Add deposit pre-auth
Wallet + transactions	KEEP	Useful for balances / refunds	Optional cash/wallet
Discovery (home/search/detail/filters)	KEEP	Maps to proximity discovery	Needs live data feed
Map screen	KEEP	Reusable shell	Add real Maps key + live markers
Booking wizard (dates→pricing→pay)	KEEP	Direct reuse for Steps 1 & 3	Add Step 2 add-ons; on-demand mode
Pre/post inspection + photo upload	KEEP	Maps to 4-photo Digital Walkaround	Enforce 4 labeled shots
Trip start/end/extend lifecycle	KEEP	Skeleton reusable	Re-bill per-min/hr; gate on IoT
Disputes (file + resolve/escalate)	KEEP	Foundation for claims	Add cross-user flag + deposit deduct
Filament admin (users/cars/bookings…)	KEEP	Back-office base	Extend into Control Tower
Reviews / ratings	KEEP	Optional but harmless	Lower priority
Push notifications + device tokens	KEEP	Alerts / comms	Reuse for theft/late alerts
Settings (currency/units/locale)	KEEP	Already app-wide	Add real translations
Reverb WebSockets	KEEP	Powers live map	Connect telemetry
Host app screens (dashboard, add_car…)	REPURPOSE	No external hosts; operator manages fleet	Become operator / fleet admin UI
Host roles / host verification	REPURPOSE	Operator is the only “host”	Collapse into admin / ops roles
Instant / request booking	REPURPOSE	Target is on-demand “take now” / slot	Simplify booking modes

Drivly capability	Decision	Why	Notes
Car model (host-owned listing)	REPURPOSE	Must become operator fleet vehicle	Add IMEI, telemetry, zone; drop host_id
Host↔renter chat	REPURPOSE	No host counterpart	At most an optional support channel
Host payouts / earnings	REMOVE	Operator owns all revenue — no payouts	Earning model N/A
Daily-only billing assumptions	REMOVE	Replaced by per-min/hr free-floating	Keep daily as one tariff option

4.7 Where we stand — pillar rollup

A directional read of progress toward the target product. This view swaps the cross-cutting pillar for a "business-model fit" pillar.

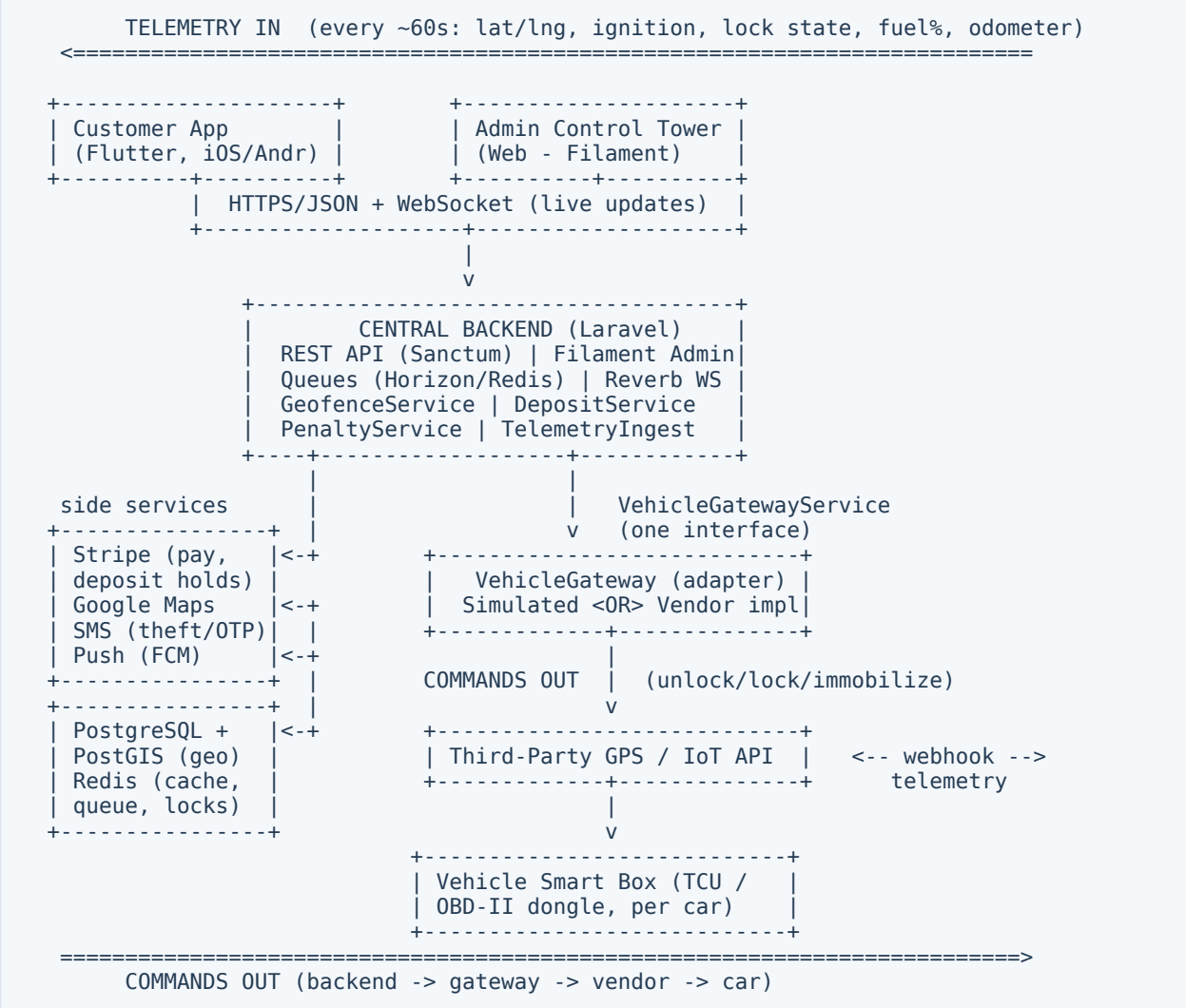
Pillar	% toward target	One-line reason
Customer Mobile App	~50%	Auth, KYC, booking, inspection, map shell, settings reuse well; keyless IoT, proximity, deposit, add-ons, per-min billing, i18n missing
Web Admin / Control Tower	~30%	Filament base exists; live map, geofence editor, tariff/penalty matrix, maintenance logs, telemetry missing
Central Backend + IoT/Geo	~25%	Strong Laravel/payments/auth/queues/WS base; no IoT gateway, no PostGIS, no telemetry, no theft/BLE fail-safes
Business-model fit	~40%	Built as P2P; needs re-aiming to operator-owned zero-human fleet (the single biggest gap)
Overall	**~35–40%**	**Half the supporting platform is a real head start; the differentiating telematics + geofencing + on-demand core is still to build**

5. Target Architecture, Data Model & MVC

This section is the bridge from “what the client asked for” to “where each piece of code lives.” It describes the target system — the keyless, operator-owned platform — and maps every capability onto a concrete model, controller, service, and screen. Where Drivly already has the right class and only the internals change, that is called out so the team reuses rather than rewrites.

5.1 System Architecture at a Glance

Four tiers: the Flutter app in the renter’s hand, the Laravel API and its background workers, the data stores (PostgreSQL + PostGIS, Redis, object storage), and the outside world the platform talks to — the IoT vehicle gateway, the payment processor, and the maps provider.



Key points:

- **The app never talks to a car directly.** Every unlock, lock, or status request goes phone → backend → gateway. The backend is the single place that authorizes a command, checks the renter is within 15 m, writes an audit row, and only then forwards it.

- **A 60-second poller and a vendor webhook** keep each car's live location, fuel, ignition, and lock state fresh in the database; Laravel Reverb pushes those updates to the map over websockets.
- **PostGIS does the geography** — "is the car inside a drop-off zone," "has it left the operating area" — as indexed spatial queries rather than hand-rolled distance maths.
- **Everything hardware-facing sits behind one interface** (the VehicleGateway, §5.2) so the entire stack can be built and tested against a simulator before a single physical device exists.

5.2 The VehicleGateway Adapter — the Hardware Seam

This is the most important design decision in the build. The client has not yet chosen an IoT vendor, and the vehicle hardware is not yet in hand. Rather than let that block everything, all hardware actions are hidden behind **one narrow interface**. The application is written against the interface; the real vendor is plugged in later as a one-class swap.

```
interface VehicleGateway
  unlock(vehicle)             -> CommandResult { accepted, commandId, latencyMs }
  lock(vehicle)               -> CommandResult
  immobilize(vehicle)         -> CommandResult   // cut starter relay (anti-theft)
  releaseImmobilizer(vehicle) -> CommandResult
  getStatus(vehicle)          -> Telemetry { lat, lng, ignition, locked,
                                             fuelPct, odometer, at }
```

Two implementations of the same contract:

Method	SimulatedGateway (build & test now)	VendorGateway (wired at M4)
unlock / lock	Flips in-memory lock state, returns success after a short fake latency, emits a telemetry event.	Calls the vendor's REST/MQTT endpoint, awaits the device ACK, maps it to CommandResult.
immobilize	Sets a flag that blocks a simulated ignition-on; used to rehearse the anti-theft flow.	Sends the starter-relay cut command to the telematics unit.
getStatus	Returns a moving GPS track, draining fuel, and toggled ignition from a scripted scenario.	Reads the live device record (last webhook / poll) and normalizes it.
telemetry	A timer emits synthetic position/fuel events so the live map and geofence logic run for real.	Vendor webhook or poll feeds the same normalized TelemetryEvent.

> **Drivly already has the seed of this**
>
> The prototype screen `gps_control_screen.dart` and its `gps_device_provider.dart` already simulate a moving vehicle and a device that responds to commands. That throwaway prototype is the proof the simulator approach works — it gets promoted from a UI toy into a backend `SimulatedGateway` that the whole platform exercises.

How a telemetry reading actually lands in the database and fans out to the map and the geofence checks:

```
  (A) Vendor WEBHOOK  -->  POST /webhooks/telemetry  --+
                                                       +--> normalize --> TelemetryEvent
  (B) 60s POLLER (queued job) --> gateway.getStatus() -+        |
```

```
                                                  v
                        writes vehicle live fields, runs GeofenceService,
                        pushes to Reverb (live map), checks pre-lock rules
```

5.3 Geofencing & Spatial Logic (PostGIS)

> **Status: MISSING in Drivly — net-new capability**
>
> Drivly stores a single lat/lng per listing and draws a pin. It has no concept of a zone, no spatial index, and no rule engine. Free-floating car-sharing lives or dies on this, so it is built fresh on PostGIS — the spatial extension for the PostgreSQL database Drivly already runs.

Three kinds of zone, one table, polygon geometry:

Zone type	What it means	Drives
Operational	The business boundary the fleet may roam — e.g. “inside Casablanca city limits.”	Out-of-bounds alerts; eligibility to start/return.
Drop-off	Approved places a rental may be ended (legal parking the operator has cleared).	“Valid return?” gate at lock time.
No-go	Carve-outs inside the operating area where the car must never be left or driven.	Blocks return; raises alert if entered.

The three spatial questions, expressed as indexed PostGIS queries:

```
1. VALID RETURN?      ST_Contains(dropoff_zone.geom, car.point)        -> true/false
2. PROXIMITY UNLOCK   ST_Distance(phone.point, car.live_point) <= 15m  -> gate the button
3. OUT-OF-BOUNDS      NOT ST_Contains(operational_zone.geom, car.pt)   -> raise alert
                      OR  ST_Contains(no_go_zone.geom, car.point)
```

5.4 Backend MVC Map (Laravel)

Every target capability mapped to its Model, the Controller that exposes it, the Service that holds the rules, and its build status. **Green** = the class exists in Drivly and is reused; **amber** = exists but needs real changes; **red** = net-new.

Capability	Model	Controller	Service / Job	Status
Auth, OTP, social login	User	AuthController	OtpService	PARTIAL
KYC document upload + review	KycDocument	KycController	KycReviewService	PARTIAL
Card tokenization & charges	Payment	PaymentController	PaymentGateway	PARTIAL
Security deposit hold / capture / release	Deposit	DepositController	DepositService	MISSING
Proximity map of cars	Vehicle	VehicleMapController	TelemetryService	PARTIAL
Tariffs (hourly / daily / seasonal)	Tariff	TariffController	PricingService	MISSING
Booking + add-ons	Booking, RentalAddon	BookingController	BookingService	PARTIAL
Walkaround inspection (4	Inspection	InspectionController	—	PARTIAL

Capability	Model	Controller	Service / Job	Status
photos)				
Proximity-gated unlock	Rental, VehicleCommand	UnlockController	GeofenceService	MISSING
Unlock / lock / status commands	VehicleCommand	VehicleCommandController	VehicleGateway	MISSING
Telemetry ingest (webhook + poll)	TelemetryEvent	TelemetryWebhookController	PollTelemetryJob	MISSING
Geofencing (zones, rules)	GeoZone	GeoZoneController	GeofenceService	MISSING
Minute / hour / day billing	Rental	—	BillingService	MISSING
Late return / penalties	PenaltyRule	—	PenaltyService	MISSING
Anti-theft / immobilize	VehicleCommand	TheftController	VehicleGateway	MISSING
Cross-user damage dispute	Dispute	DisputeController	DisputeService	PARTIAL
Maintenance scheduling	MaintenanceLog	MaintenanceController	—	MISSING
Fleet status (Control Tower)	Vehicle	Admin\FleetController	TelemetryService	MISSING
Live websocket fan-out	—	(Reverb channels)	BroadcastTelemetry	MISSING
Ratings & reviews	Review	ReviewController	—	DONE
Notifications (push / SMS / email)	Notification	—	NotificationService	PARTIAL

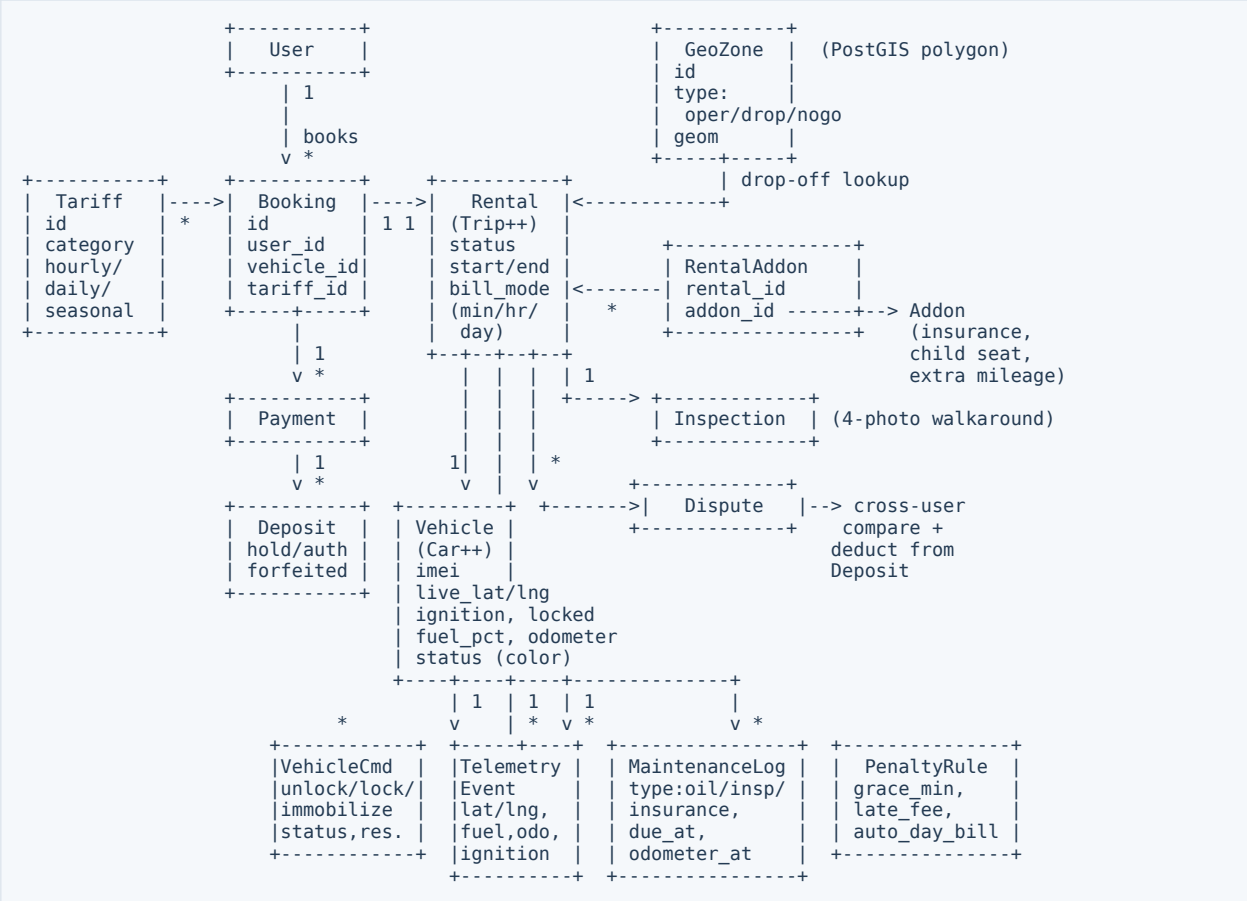
5.5 Mobile Map (Flutter, Clean Architecture)

Drivly's app already follows a feature-first clean architecture (data / domain / presentation per feature). Target capabilities slot into that same shape — most are new screens and use-cases inside existing or sibling feature folders.

Feature area	Key screens / widgets	Domain use-cases	Status
onboarding / auth	Login, OTP, Social sign-in	SignIn, VerifyOtp	PARTIAL
kyc	ID + License capture, status	SubmitKyc, WatchKycStatus	MISSING
payment	Add card, wallet, deposit consent	TokenizeCard, AuthorizeDeposit	PARTIAL
discovery / map	Proximity map, car detail sheet	WatchNearbyCars, GetCar	PARTIAL
booking	Vehicle + duration, add-ons, pay	CreateBooking, PriceQuote	PARTIAL
navigation	Pedestrian route to car	RouteToCar	PARTIAL
unlock / control	Walkaround, proximity-gated Unlock/Lock	Unlock, Lock, CheckProximity	MISSING
trip (live)	In-rental status, live meter, fuel	WatchRental, EndRental	PARTIAL
return	In-zone check, return + lock	ValidateReturn, ReturnCar	MISSING
theft / safety	Report, immobilized state UI	ReportIssue	MISSING
trips history	Past rentals, receipts, disputes	ListTrips, OpenDispute	DONE
profile	Account, documents, payment methods	GetProfile	DONE

5.6 Data Model (Entity-Relationship)

The target schema. Boxes marked `(Car++)` and `(Trip++)` are Drivly's existing Car and Trip tables extended with new columns rather than replaced; the rest — GeoZone, VehicleCommand, TelemetryEvent, Deposit, PenaltyRule, MaintenanceLog — are net-new.



> **New columns on the existing Vehicle (Car) table**
>
> `imei` (telematics unit), `live_lat / live_lng`, `ignition`, `locked`, `fuel_pct`, `odometer`, and a `status` colour (available / in-rental / maintenance / theft). These are what the live map and the Control Tower read.

5.7 IoT Command & Telemetry Contracts

The concrete request/response shapes between the backend and the gateway. These are written against the simulator first; the vendor adapter must satisfy the same contracts. Each command is authorized, audited, and idempotent on a command id.

Unlock

Issued only after the renter passes the proximity gate and the walkaround. The backend writes a VehicleCommand row, calls the gateway, and waits for the device ACK before telling the app the doors are open.

```
Req : { "vehicle_id":"v_123", "rental_id":"r_88", "phone_lat":33.59, "phone_lng":-7.61 }
Res : { "accepted":true, "command_id":"cmd_5f", "lock_state":"unlocked", "latency_ms":820 }
Consequence: proximity (<=15m) re-verified -> Rental -> InTrip, counter starts,
             app shows "retrieve key from glovebox", VehicleCommand logged.
```

Lock

Issued at end-of-rental, but only after the pre-lock checks pass (in a drop-off zone, ignition off, doors closed). A successful lock stops the billing meter.

```
Req : { "vehicle_id":"v_123", "rental_id":"r_88" }
Res : { "accepted":true, "command_id":"cmd_60", "lock_state":"locked" }
Consequence: only allowed after pre-lock checks pass -> counter stops, invoice
             calculated (incl. penalties), deposit hold released or partially
             captured, vehicle returns to map as Available.
```

Status / Telemetry poll

The 60-second heartbeat. Normalized into a TelemetryEvent, it refreshes the car's live fields, feeds the map, and runs the geofence and pre-lock rules.

```
Res : { "vehicle_id":"v_123","lat":33.5912,"lng":-7.6101,"ignition":"off",
        "locked":true,"fuel_pct":62,"odometer":48213,"at":"2026-06-26T10:00:00Z" }
Consequence: writes TelemetryEvent, updates live admin map color, runs geofence
             check, feeds maintenance odometer flags.
```

Immobilize (anti-theft)

Cuts the starter relay so the engine cannot be restarted. Triggered by the operator from the Control Tower, or automatically on a theft signal (e.g. movement with no active rental).

```
Req : { "vehicle_id":"v_123", "reason":"out_of_bounds" }
Res : { "accepted":true, "command_id":"cmd_61", "engine":"immobilized" }
Consequence: vehicle cannot restart, high-priority admin alert raised, SMS to ops.
```

Telemetry event (push)

The vendor webhook variant — the same normalized payload the poller produces, arriving event-driven instead of on a timer.

```
POST /webhooks/telemetry
Body: { "imei":"86xxxxxx","ts":"2026-06-26T10:01:00Z","gps":{"lat":33.59,"lng":-7.61},
        "ign":1,"lock":0,"fuel":61,"odo":48230 }
Consequence: normalize -> TelemetryEvent -> update Vehicle live fields ->
             GeofenceService.check() -> if out-of-bounds w/o active rental -> theft mode.
```

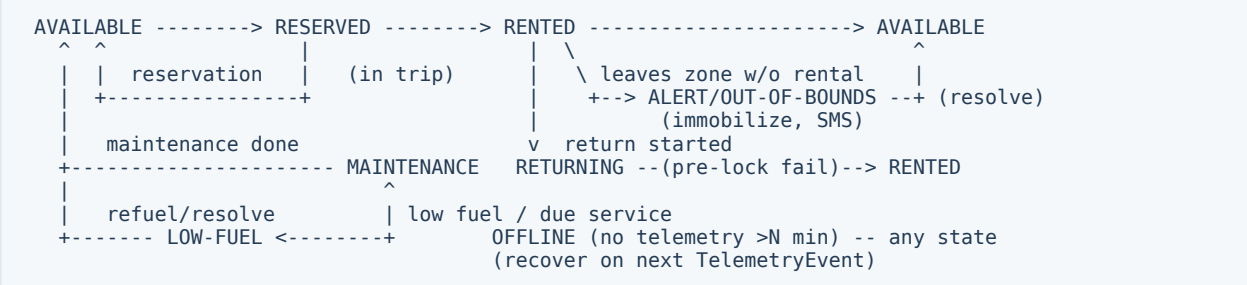
5.8 State Machines

Two lifecycles govern the platform. Keeping them explicit is what makes edge cases (Section 6) tractable — every scenario is just a path, or a blocked transition, through one of these.

Vehicle lifecycle

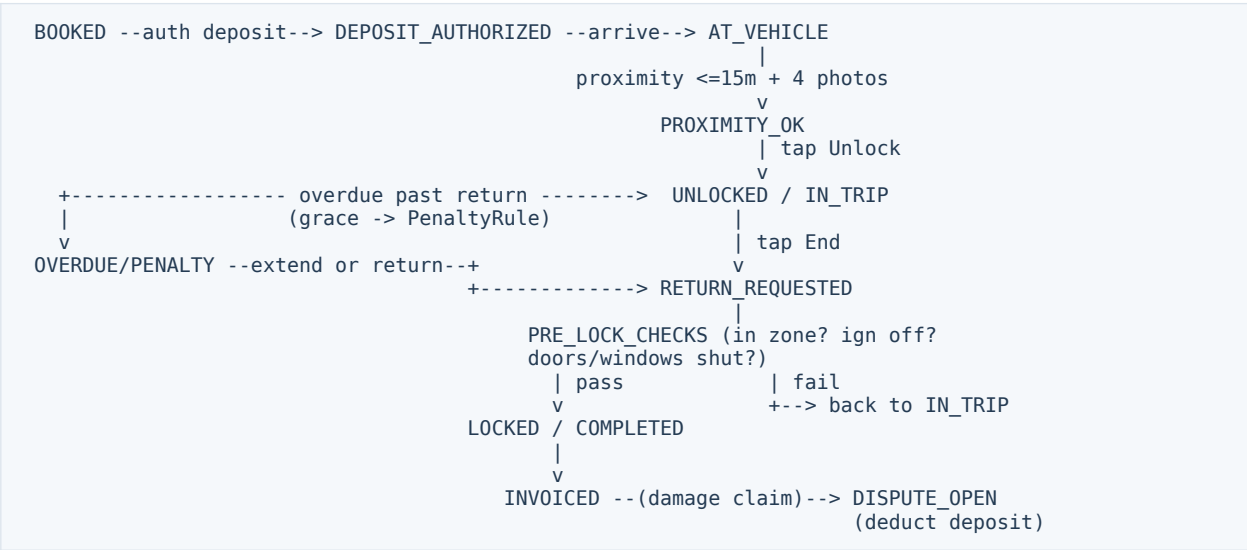
What colour a car shows on the map and whether it can be booked at all.

```
            reserve           unlock             lock(valid return)
```



Rental lifecycle

From booking to settled, including the deposit hold and the penalty / dispute branches.

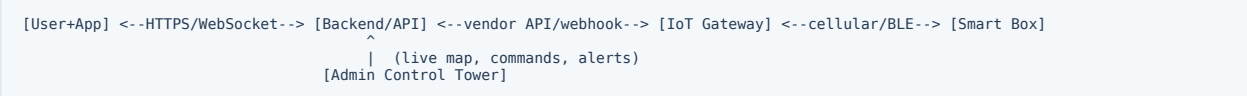


6. End-to-End Flows

These walk the platform the way a person actually meets it — in plain words first, then as numbered steps, then as a diagram. They describe the **target** experience (keyless, staff-free), not Drivly's current peer-to-peer flow. Section 6.1 is the happy path; 6.2–6.13 are the edge cases that decide whether the product feels trustworthy. Every one is a path — or a deliberately blocked transition — through the state machines in §5.8.

The five actors and how they connect:

Actor	Role in every flow below
User	The renter, holding the phone. Never touches a physical key.
App	Flutter client. Shows the map, runs the booking wizard, gates the unlock button on GPS distance, captures the walkaround.
Backend	Laravel API + workers. The only actor allowed to authorize a command, charge, hold a deposit, run geofence rules, and start/stop the meter.
IoT / Smart Box	The in-car telematics unit behind the VehicleGateway. Executes unlock / lock / immobilize and streams telemetry.
Admin	The operator at the Control Tower — watches the live fleet, approves KYC, handles theft, disputes, and maintenance.



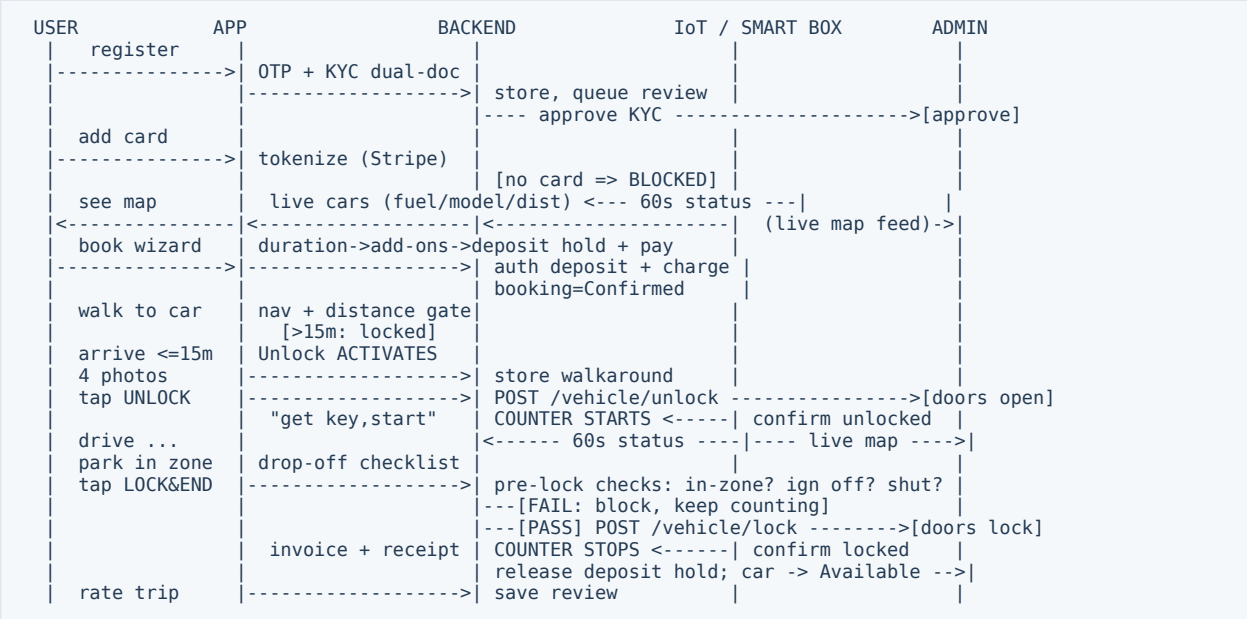
6.1 The Happy Path — Register to Return

Everything works: the renter signs up, gets approved, books a nearby car, walks to it, unlocks it, drives, parks in an approved zone, and ends the rental. This is the spine; every later scenario is a deviation from one of these steps.

1. Register with email / phone / social, then verify the phone by OTP.
2. Complete KYC — upload Government ID and Driver License (two documents). Backend queues them; admin (or an automated check) approves.
3. Add a payment method; the card is tokenized. No valid card on file means no booking — a hard block.
4. Open the map: nearby cars appear with live fuel level, model, and walking distance, refreshed by the 60-second telemetry feed.
5. Booking wizard — pick the car and rental duration; add options (Premium Insurance, Child Seat, Extra Mileage); authorize the security-deposit hold and pay upfront. Booking becomes Confirmed.
6. Walk to the car using pedestrian navigation; the app shows live distance to the car's GPS.
7. The Unlock button stays disabled until the phone is within 15 m of the car.
8. Within range, complete the 4-photo Digital Walkaround (front, back, left, right); photos are stored as the pre-rental condition record.

9. Tap Unlock → backend authorizes → POST /vehicle/unlock → doors open. The billing meter starts on confirmed unlock.
10. Drive. Telemetry streams position and fuel every 60 seconds to the live map and the operator.
11. To finish, park inside an approved drop-off zone and tap Lock & End.
12. Backend runs pre-lock checks — in a drop-off zone? ignition off? doors and windows shut? If any fail, the lock is blocked and the meter keeps running with on-screen guidance.
13. All checks pass → POST /vehicle/lock → doors lock → meter stops; invoice and receipt are produced.
14. Backend releases the deposit hold and flips the car back to Available.
15. Optionally rate the trip.

The same flow as a swimlane across all five actors:



6.2 Booking Blocked — No KYC or No Payment

The guard rail that protects the operator. A renter cannot reach the booking wizard until identity is verified and a valid card is on file. The block is explicit, with a clear path to fix it.

1. User taps Book.
2. Backend checks: is KYC approved? If not, show KYC status / prompt re-upload and stop.
3. If approved, check: is there a valid payment method on file? If not, prompt “Add payment method” and stop.
4. Only when both pass does the booking wizard open.

```
                Open booking wizard
```

6.3 Payment or Deposit Declined

The card fails at the pay step — insufficient funds, a bank block, or the deposit hold is refused. The car must stay free for someone else, and the renter gets a precise reason and a retry path.

1. At the pay step, backend authorizes the deposit hold and charges the rental.
2. If declined, the car stays Available and the app shows “Payment failed: <reason>.”
3. User can try another card (retry) or cancel back to the map.
4. No booking is created and no hold persists until a charge succeeds.

```
[Pay step] --> auth deposit + charge --ok--> booking=Confirmed
                      |declined
                keep car Available; "Payment failed: <reason>"
                      |
                [Try another card] --> retry  |  [Cancel] --> back to map
```

6.4 Renter Not Within 15 m of the Car

Stops someone unlocking a car they are nowhere near. The unlock button is governed by live distance between the phone’s GPS and the car’s GPS, not by a manual override.

1. App continuously compares phone GPS to the car’s live GPS.
2. If distance is greater than 15 m, the Unlock button is disabled with guidance to keep walking.
3. Once within 15 m, Unlock is enabled and the renter proceeds to the 4-photo walkaround.

```
phone GPS  vs  car live GPS  --> distance > 15m? --yes--> [Unlock DISABLED] + guidance
                                        |no
                                [Unlock ENABLED] --> proceed to 4-photo walkaround
```

6.5 Unlock Command Fails or Times Out

The renter is in range and taps Unlock, but the car does not confirm — weak signal, a slow device, or a gateway hiccup. The renter must never be billed for a car they could not open.

1. App sends Unlock; backend issues POST /vehicle/unlock.
2. If the device confirms, the meter starts.
3. If it times out, the backend retries a bounded number of times.
4. If a retry confirms, the meter starts on confirmation.
5. If it keeps failing, show “Contact support,” raise an admin alert, and start no billing.

```
[Unlock] --> POST /vehicle/unlock --> confirmed? --yes--> COUNTER STARTS
                                            |no (timeout)
                                      retry (x N) --confirmed--> COUNTER STARTS
                                            |still failing
                                      "Contact support" + Admin alert ; NO billing
```

6.6 Dead Zone — No Cellular at the Car

The car sits in an underground garage or a signal black spot. A keyless platform needs a fallback so the renter is not stranded — ideally Bluetooth phone-to-car, with a manual code as a last resort.

1. Backend tries the normal IoT path; checks whether the car is reachable over cellular.
2. If not reachable, fall back to BLE where supported: the phone talks directly to the Smart Box to unlock / lock.
3. If BLE is unavailable, present an offline code plus toll-free verification.
4. Reconcile the command and billing state once the car is back online.

```
command --> cellular reachable? --yes--> normal IoT path
                |no
           BLE supported? --yes--> phone <--BLE--> Smart Box (unlock/lock)
                |no
           show offline code + toll-free verification ; reconcile when back online
```

6.7 Pre-Lock Check Fails at Return

The renter tries to end the rental but something is wrong — still parked outside an approved zone, engine running, or a door open. The car must not lock (and the meter must not stop) until the car is genuinely safe and legal to leave.

1. User taps Lock & End.
2. Backend checks: in an approved zone? If not, “Move into zone.”
3. Ignition off? If not, “Turn off engine.”
4. Doors and windows shut? If not, “Close the door / window.”
5. While any check fails the lock is blocked and the meter keeps running; when all pass, POST /vehicle/lock, the meter stops, and the invoice is issued.

```
[Lock & End] --> in-zone? --no--> "Move into zone"      -+
                 ign off? --no--> "Turn off engine"      |--> BLOCKED, counter RUNS
                 doors/windows shut? --no--> "Close door"+
                      | all pass
                 POST /vehicle/lock --> counter STOPS, invoice
```

6.8 Return Attempted Outside Any Zone

A specific, common case of the pre-lock failure: the renter is done but standing entirely outside the operating area. Policy decides whether they must drive back or may end where they are for a fee.

1. Backend detects the car is outside all drop-off / operational zones.
2. Offer navigation to the nearest valid zone, or — if policy allows — End with an out-of-zone fee.
3. If they drive back, the normal return (6.1) applies.
4. If they end out-of-zone, the meter stops and the fee is added to the invoice.

```
outside all zones? --yes--> [Nav to nearest zone]  OR  [End + out-of-zone fee*]
                                  |                         (*if policy allows)
                            return in zone (6.1)      counter stops, fee added to invoice
```

6.9 Late Return & Penalty

The scheduled end passes. The system gives a short grace window and an easy way to extend; if neither is used, it bills the next full day and flags the rental — automatically, without staff chasing the renter.

1. At the scheduled end, check whether the renter is inside the grace window.
2. If they extend in the app, set a new end time with no penalty.
3. If the grace window expires with no extension, auto-bill the next full day and apply an admin penalty.
4. Notify the user and mark the rental overdue.

```
scheduled end --> within grace window?
       |                          |
  extend in app? --yes--> new end time, no penalty
       |no                        |grace expired, no extend
       +----------------> AUTO: next full day + admin penalty --> notify user, mark overdue
```

6.10 Theft / Unauthorized Movement

A car moves with no active rental, leaves the operating area, or crosses a border. This is the platform's most serious automated response: alert the operator and arm the engine immobilizer.

1. The 60-second telemetry check finds movement, out-of-zone, or border-crossing with no active rental.
2. Raise a high-priority admin alert, flag the car RED, and SMS the ops team.
3. Arm POST /vehicle/immobilize; on admin confirmation, cut the engine so it cannot restart and escalate (e.g. alert police).

```
60s status --> active rental? --yes--> normal tracking
                    |no
              moving / out-of-zone / border crossed?
                    |yes
       HIGH-PRIORITY admin alert (car=RED) + SMS to ops
                    |
       ARM POST /vehicle/immobilize --> [admin confirms] --> engine cut, cannot restart, alert police
```

6.11 Cross-User Damage & Deposit Forfeiture

User B starts a rental and reports damage at the walkaround. The platform must work out whether the previous renter, User A, caused it — using the photo trail — and, if so, charge A rather than the innocent B.

1. User B's walkaround / report flags damage; the system identifies the previous driver, User A.
2. Build a case: A's post-rental photos versus B's pre-rental photos.
3. Admin reviews. If A is at fault, deduct the repair fee from A's deposit and release the rest.
4. If not, release A's deposit with no charge; B is never charged for pre-existing damage.

```
User B walkaround/report (damage) --> system finds previous driver = User A
                    |                                  |
         build case: A post-photos  vs  B pre-photos
                    |
         ADMIN reviews --> A at fault? --yes--> deduct fee from A's deposit; release rest
                                        --no--> release A's deposit; no charge
```

6.12 Low Fuel / Maintenance Auto-Flag

Telemetry notices a car is low on fuel or due for service. It quietly pulls the car off the customer map so no one books a vehicle that can't finish a trip, and tells ops to act.

1. The 60-second check sees low fuel / battery, or odometer at/over the service threshold.
2. Set the car's status to Orange (Maintenance / Low-Fuel) and remove it from the customer map.
3. Notify ops.
4. Once refuelled / serviced, return the car to Available (Green).

```
60s status --> low fuel/battery OR odometer >= service threshold?
                    |yes
       set status=Orange (Maintenance/Low-Fuel) --> remove from customer map --> notify ops
                    | serviced/refuelled
              back to Available (Green)
```

6.13 Cancellation & No-Show

Plans change before pickup, or the renter simply never shows. The booking and the deposit hold must unwind cleanly, and the car must be freed for others, on a clear policy.

1. Booking is Confirmed. If the user cancels before unlock, refund per policy, release the deposit hold, and free the car.
2. If the pickup window passes with no unlock, auto-expire the booking.
3. On expiry, release the hold or apply a no-show fee per policy.
4. Otherwise the renter proceeds to pickup (6.1).

```
booking Confirmed --> user cancels before unlock? --yes--> refund per policy + release hold + free car
                          |no
                     pickup window passed without unlock? --yes--> auto-expire
                          |                                          (release hold or no-show fee)
                     proceed to pickup (6.1)
```

6.14 Scenario Coverage — Reuse vs New Work

What each flow can borrow from Drivly versus what is genuinely new, with a rough effort weight (S / M / L / XL — see the legend in Section 8.2).

#	Scenario	Reuses from Drivly	New work	Effort
6.1	Happy path	Auth, booking wizard shell, payments, photos, ratings	Proximity gate, unlock/lock, meter, deposit, telemetry map	**XL**
6.2	Booking blocked (KYC/pay)	KYC + payment models, booking entry	Hard-block gate before wizard	**S**
6.3	Payment / deposit declined	Payment gateway, error surfacing	Deposit authorize + retry logic	**M**
6.4	Not within 15 m	Map, live GPS, car detail	Distance gate on Unlock button	**M**
6.5	Unlock fails / timeout	—	Command retry, no-bill, admin alert	**M**
6.6	Dead zone (no cellular)	—	BLE fallback + offline code path	**L**
6.7	Pre-lock check fails	—	In-zone / ignition / door rules + meter	**M**

#	Scenario	Reuses from Drivly	New work	Effort
			hold	
6.8	Return outside zone	Navigation	Out-of-zone policy + fee	**M**
6.9	Late return & penalty	Notifications	Grace window, extend, auto day-bill, penalty	**M**
6.10	Theft / unauthorized move	—	Geofence watch, immobilize, ops SMS	**L**
6.11	Cross-user damage	Inspection photos, dispute shell	Photo A/B case-build + deposit deduct	**M**
6.12	Low fuel / maintenance	—	Threshold watch, status flip, map hide	**S**
6.13	Cancellation / no-show	Booking lifecycle	Auto-expire, hold release, no-show fee	**S**

7. Feasibility — Can This Be Built?

Verdict: YES — and ~90–95% of it can be built and tested before any hardware arrives

Drivly already supplies the hard parts that usually sink a project early: a live Flutter app, Laravel + PostgreSQL, real auth, KYC, payments, photos, ratings, and clean architecture on both ends. The new work is well-understood (geofencing, a command/telemetry layer, on-demand billing). Because every hardware action sits behind one interface, the team builds against a simulator from day one and the single real dependency — the vendor device — is isolated to the back half of the plan.

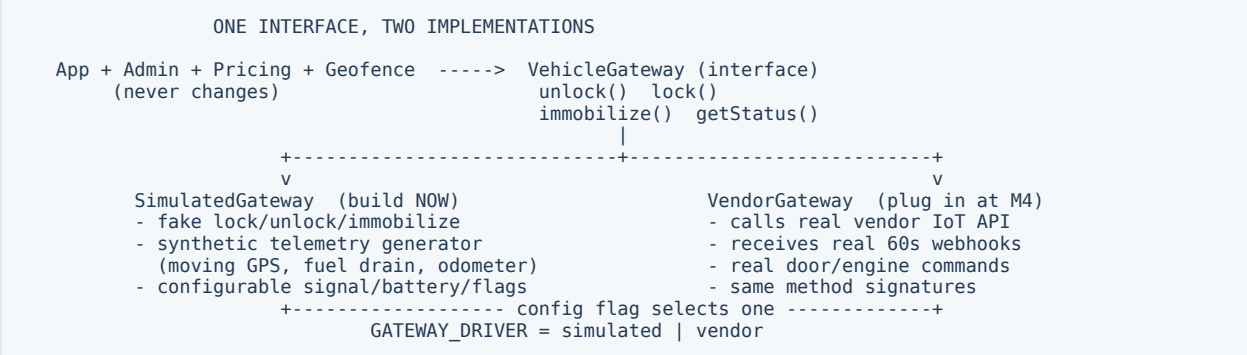
7.1 Capability-by-Capability Feasibility

Read the middle column as: **Yes** = build and fully verify now; **Yes – sim** = build now against the simulator, final sign-off needs the device; **No – device** = genuinely needs hardware in hand.

Capability	Buildable?	How we de-risk it
Auth, OTP, social login	Yes	Extend Drivly's existing auth; OTP via the chosen SMS provider.
KYC document capture & review	Yes	Drivly already uploads documents; add the dual-doc + review workflow.
Payments & card tokenization	Yes	Drivly has a payment layer; swap/confirm the Morocco processor (CMI / Stripe).
Security deposit hold / capture	Yes	Standard pre-auth on the processor; no hardware involved.
Proximity map of cars	Yes – sim	Map is built now; live positions come from the simulator until devices stream real GPS.
On-demand billing (min/hr/day)	Yes	Pure backend rules over the rental timeline; fully testable now.
Geofencing & zones (PostGIS)	Yes	Spatial queries on PostgreSQL; test with seeded zones and simulated tracks.
Booking wizard + add-ons	Yes	Rework Drivly's booking flow; no hardware dependency.
4-photo walkaround	Yes	Drivly already captures photos; wire them as pre/post condition records.
Proximity-gated unlock logic	Yes – sim	Distance gate and command flow built now; the actual door-open is simulated.
Unlock / lock / immobilize	Yes – sim	Full command path, retries, audit built against SimulatedGateway.
Telemetry ingest (webhook/poll)	Yes – sim	Ingest pipeline + normalization built now; fed by synthetic events.
Real door / engine actuation	No – device	Needs the vendor unit and API; validated at integration (M4).
Real GPS / fuel accuracy	No – device	Depends on the device's sensors; only the device can confirm fidelity.
BLE offline fallback	No – device	Phone-to-box Bluetooth can only be proven on the real hardware.
Control Tower live fleet view	Yes – sim	Dashboard + websocket fan-out built now on simulated telemetry.

7.2 The Simulator Strategy — Why Hardware Doesn't Block the Build

One interface, two implementations, selected by a single config flag. The application, admin, pricing, and geofence layers are written once and never change when the real vendor is plugged in.



What this buys the project:

- Roughly 90–95% of the system is built and tested before a device exists — no idle waiting on procurement.
- Edge cases that are dangerous or impossible to stage physically (theft, dead zones, unlock timeouts, low fuel) are rehearsed deterministically by scripting the simulator.
- When the vendor is chosen, only one class — VendorGateway — is implemented against the same signatures, shrinking integration to wiring and field validation rather than rebuilding features.
- If the vendor changes later, the blast radius is one adapter, not the whole codebase.

7.3 What Genuinely Needs Hardware

The short, honest list of work that cannot be finished without the physical unit and the vendor API. All of it concentrates in the integration milestone (M4).

Item	Why hardware is required	When
Real door lock / unlock	Only the device actuates the physical locks.	**M4**
Engine immobilization	Cutting the starter relay is a hardware command.	**M4**
GPS / fuel accuracy	Real-world sensor fidelity can only be measured on the unit.	**M4**
Live telemetry timing	Actual webhook cadence and latency come from the vendor.	**M4**
BLE offline fallback	Phone-to-box Bluetooth must be proven on the device.	**M4**

7.4 Non-Hardware Items the Client Must Supply

These are not engineering risks — they are inputs and accounts only the client can provide. Gathering them early keeps the build unblocked.

Item	What’s needed	Blocks
Payment processor	Choose CMI (local) or Stripe; provide merchant credentials.	Live payments & deposits
SMS / OTP provider	Account + sender ID for OTP and ops alerts.	Verification, theft SMS
Maps API key	Google Maps (or equivalent) billing-enabled key.	Map, pedestrian nav, distance
App store accounts	Apple Developer ($99/yr) and Google Play ($25 once).	Public release

Item	What's needed	Blocks
PostGIS hosting	A managed PostgreSQL with PostGIS enabled.	Geofencing in production
Localization (AR / FR)	Arabic + French strings and RTL review for the Morocco market.	Launch readiness
Legal & insurance	Rental terms, insurance product, deposit/penalty policy.	Go-live, dispute rules
Zone definitions	The actual operational, drop-off, and no-go polygons.	Geofence correctness

7.5 The Blunt Risk

The vendor is the single biggest risk — everything else is under our control

The software can be ~90–95% complete and still not ship if the IoT vendor is undecided, their API documentation is thin, or devices arrive late. The simulator removes this from the critical path for the bulk of the build, but it cannot remove it from the finish. The most valuable thing the client can do is choose the vendor and get documentation plus at least one test device into the team's hands by the end of the first week.

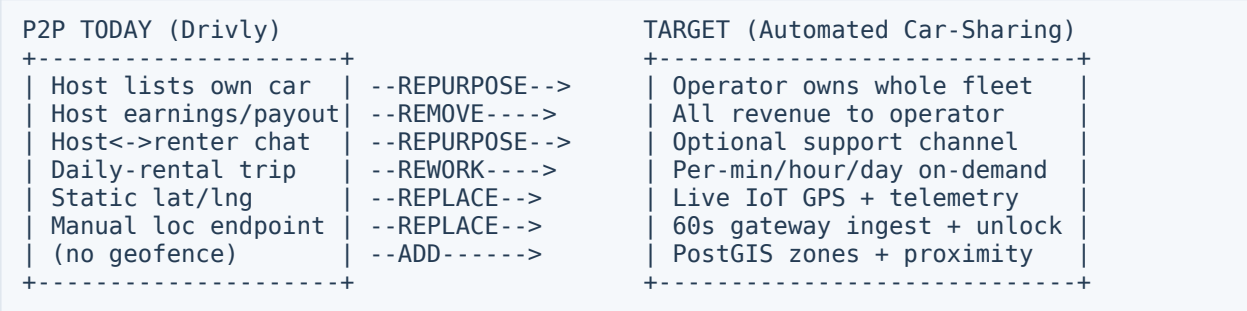
8. Roadmap, Effort & Cost

A five-milestone plan that front-loads everything buildable without hardware and isolates the vendor dependency to M4. The sequencing is deliberate: by the time real devices are needed, the entire software platform is already exercising them in simulation.

8.1 Phase Plan

M	Milestone	Focus	Weeks
M1	Foundation & Repurpose	Fork Drivly, strip host/P2P concepts, stand up auth/KYC/payments for the operator model, seed data model.	1–2
M2	Core Rental Engine	Booking wizard + add-ons, deposit hold, on-demand (min/hr/day) billing, tariffs, walkaround.	3–5
M3	IoT (Simulated) & Geofencing	VehicleGateway + SimulatedGateway, telemetry ingest, live map, PostGIS zones, proximity-gated unlock, command layer.	6–8
M4	Hardware Integration	Implement VendorGateway against the real API, validate door/engine/GPS/fuel, BLE fallback, field-test. (Hardware-gated.)	9–10
M5	Control Tower & Hardening	Admin fleet dashboard, theft/immobilize, disputes, maintenance, penalties, localization, UAT.	11–12

How the two products relate at the code level — what gets repurposed, removed, reworked, replaced, or added on the way from Drivly to the target:



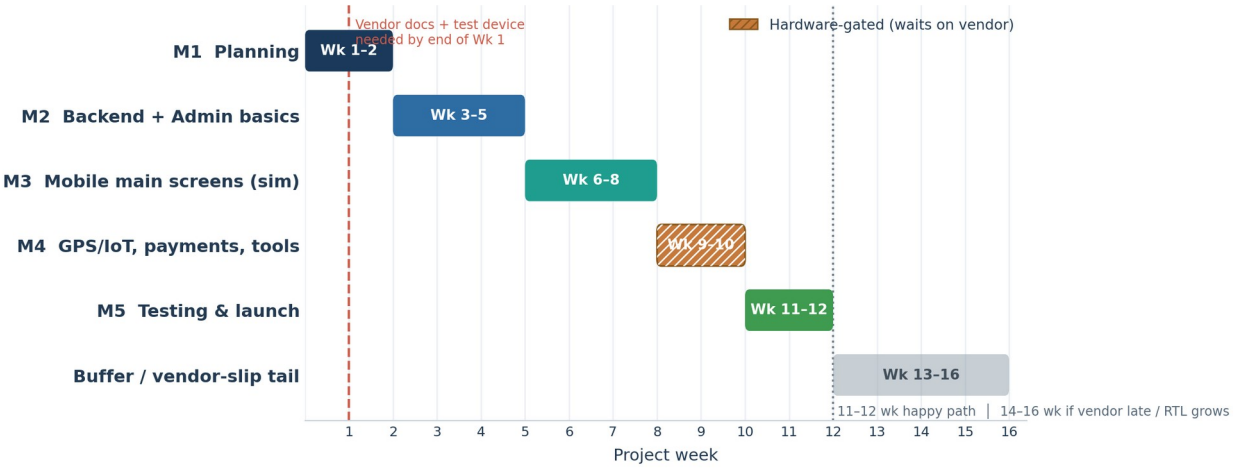
8.2 Effort by Workstream

> **Effort legend**
>
> **S** ~1–3 dev-days **M** ~4–8 dev-days **L** ~2–3 dev-weeks **XL** ~4+ dev-weeks

Workstream	Effort	M	Note
Repurpose Drivly (strip P2P, operator model)	L	M1	Mostly deletion + rewiring ownership.
Auth / OTP / social	S	M1	Extend existing.
KYC dual-doc + review	M	M1	Build on Drivly's uploads.
Payments + deposit hold	M	M1–2	Confirm Morocco processor.
Tariffs + on-demand billing	L	M2	Net-new pricing engine.

Workstream	Effort	M	Note
Booking wizard + add-ons	M	M2	Rework existing flow.
Walkaround inspection	S	M2	Reuse photo capture.
VehicleGateway + SimulatedGateway	L	M3	The hardware seam.
Telemetry ingest + live map (Reverb)	L	M3	Webhook + poll + websockets.
Geofencing (PostGIS zones + rules)	L	M3	Net-new spatial layer.
Proximity-gated unlock + command layer	L	M3	Distance gate, retries, audit.
VendorGateway (real hardware)	XL	M4	Hardware-gated; vendor-dependent.
Control Tower (fleet dashboard)	L	M5	Admin live view + actions.
Theft / immobilize / disputes / maintenance	L	M5	Operational safety net.
Penalties + late-return automation	M	M5	Grace, auto day-bill.
Localization (AR / FR / RTL) + UAT	M	M5	Market readiness.

8.3 Timeline

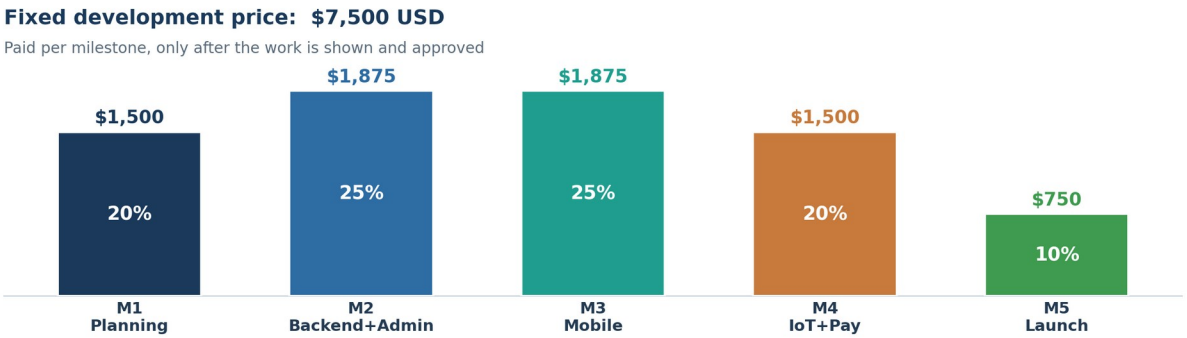


Indicative 12–16 week schedule. M4 (orange, hatched) is hardware-gated; the vendor dependency lands in Week 1.

An honest read of the schedule:

- **12 weeks is plausible only if two things hold:** the head start from reusing Drivly genuinely offsets the new work, and the IoT vendor delivers documentation and a device on time. Neither is guaranteed.
- **Plan and budget for 14–16 weeks.** The realistic range carries a buffer (Weeks 13–16) for hardware integration surprises, field testing, and UAT fixes — the parts most likely to slip.
- **M4 is the pivot.** Everything before it can run to schedule on simulation; M4's start depends entirely on the vendor. If the device is late, M4 (and only M4) slides — the rest is already done.

8.4 Cost



Fixed price of $7,500 across five milestones (20 / 25 / 25 / 20 / 10%).

Why $7,500 is lean but defensible

The figure assumes the team is extending a working codebase, not starting from zero. Drivly already covers auth, KYC, payments, photos, ratings, and clean architecture on both client and server — so the budget is spent on the genuinely new layers (geofencing, IoT command/telemetry, on-demand billing, the Control Tower) rather than rebuilding the foundations. Remove the Drivly head start and this would not be a $7,500 project.

Recurring / operating costs (the client's, not in the build price):

Item	Notes	Rough cost
IoT hardware (per car)	Telematics unit + install, per vehicle.	Vendor-quoted
SIM / cellular data	Per device, monthly.	Per device / mo
Apple Developer	Annual.	$99 / yr
Google Play	One-time.	$25 once
Maps API	Usage-based (pedestrian nav, distance).	Usage-based
SMS / OTP	Per message (verification + alerts).	Per message
Payment processing	Per-transaction fees (CMI / Stripe).	Per transaction
PostGIS hosting	Managed PostgreSQL + PostGIS.	Monthly
Domain / SSL / servers	Standard hosting and certificates.	Monthly

8.5 Client Decisions Checklist

The open decisions that gate the build, roughly in the order they bite. The first is on the critical path — the rest can be settled in parallel during M1–M2.

1. **IoT vendor — CRITICAL, Week 1.** Choose the provider; secure API docs and at least one test device. Everything hardware-final waits on this.
2. **Business-model confirmation.** Confirm operator-owned, free-floating (not peer-to-peer) so the repurpose scope is locked.
3. **Payment processor.** CMI (local) vs Stripe; provide merchant credentials.
4. **Maps provider + key.** Billing-enabled Google Maps (or equivalent).

5. **SMS / OTP provider.** Account and sender ID.
6. **Developer accounts.** Apple and Google Play.
7. **PostGIS hosting.** Managed PostgreSQL with PostGIS.
8. **Brand & app identity.** Name, icon, theme for the new product.
9. **Initial vehicle list.** The fleet to onboard and its details.
10. **Legal & insurance.** Rental terms, insurance product, deposit and penalty policy.
11. **Zone definitions.** Operational, drop-off, and no-go polygons.
12. **Languages.** Confirm Arabic + French + RTL scope.
13. **KYC policy.** Manual review, automated, or hybrid; acceptance rules.
14. **Support model.** Toll-free / in-app channel for the dead-zone and unlock-failure fallbacks.

8.6 Risks & Mitigations

Risk	Severity	Mitigation
IoT vendor undecided / late device	High	Simulator-first build; isolate to M4; push vendor selection + test device to Week 1.
Thin / changing vendor API docs	High	One narrow adapter (VendorGateway) absorbs change; contracts pinned against the simulator.
GPS / unlock unreliable in the field	Med	Retry + no-bill logic (6.5), BLE + offline-code fallback (6.6), proximity gate tuning.
Payment / deposit edge cases	Med	Confirm processor early; rehearse declines, holds, refunds, forfeiture in M2.
Geofence accuracy / zone gaps	Med	PostGIS with client-supplied polygons; seeded test tracks; operator override.
Schedule slips into hardware phase	Med	14–16 week budget with a buffer; only M4 depends on hardware timing.
Localization / RTL defects	Low	AR/FR strings + RTL review folded into M5 with UAT.
Theft / liability exposure	Med	Immobilize flow (6.10), deposit forfeiture (6.11), clear legal terms before go-live.

8.7 Recommended Next Steps

1. **Lock the IoT vendor this week** and get API documentation plus one test device to the team — this is the only true critical-path item.
2. **Confirm the operator-owned model in writing** so the M1 repurpose scope (what to strip from Drivly) is unambiguous.
3. **Stand up accounts and keys in parallel** — payment processor, Maps, SMS, Apple/Google, PostGIS hosting — so none of them blocks M1–M2.
4. **Start M1 immediately against the simulator;** there is no reason to wait for hardware to begin.
5. **Supply zone polygons and legal/insurance terms by M3,** when geofencing and go-live rules need them.
6. **Plan to 14–16 weeks,** treating 12 as the optimistic case that depends on vendor punctuality.

Bottom line

Drivly is a strong, working foundation — but it is a different product. With a disciplined repurpose, a simulator-first build behind a single hardware seam, and an early vendor decision, the automated car-sharing platform is realistic on a 12–16 week timeline at the stated $7,500 build price. The biggest lever the client controls is choosing the IoT vendor now.